# Enterprise-Scale Apex Across Multiple Packages

## Contracts, configuration and extensibility

FFLib Series · Session 006

John M. Daniel

Senior Director, Digital Platforms · Steampunk, Inc.



# About me

Senior Director of Digital Platforms, Steampunk, Inc. | Apex Enterprise Patterns Maintainer & Architect

- **Apex Enterprise Patterns (fflib) maintainer & architect** — steward of `fflib-apex-common`, `fflib-apex-mocks`, `force-di`, and `AT4DX` at github.com/apex-enterprise-patterns
- **Open source education** — promoting awareness and adoption of Domain, Service, and Selector patterns for enterprise Salesforce development
- **Senior Director, Digital Platforms** — leading Salesforce platform engineering standards and delivery at Steampunk, Inc.




When I am not working on Apex architecture, I am probably driving my Jeep on the beach.

# FFLib Series

## Promises we made

| | |
|---|---|
| Andy, Session 2: metadata-driven dependency injection, and interfaces to resolve Services | → slide **19, 24** |
| Session 4: AT4DX adds fields to a Selector from another package (field sets) | → slide **30** |
| Session 4: `@NamespaceAccessible` | → slide **16** |
| Session 5: the `WDGT_` prefix, "a namespace inside one package" | → slide **16** |
| Session 5: "AT4DX keeps a factory, on purpose" | → slide **23** |

# One package so far

*Today the same layers split across packages*

- **Sessions 1 to 5 lived in one package:** `force-app` . Every class could see every other class
- **Service, Domain, Selector and Unit of Work** don't change today. Where they live does
- **Session 5's seams** — `newInstance()` , constructors, mocks — are what make the split possible

> Same patterns. Different boxes. The boxes are packages.

# Today

| TODAY | |
|---|---|
| Meridian: one org, four divisions | ☐ |
| The package layers | ☐ |
| Cross-package architecture | ☐ |
| Shared contracts and interfaces | ☐ |
| Configuration-driven design | ☐ |
| Extensibility patterns | ☐ |

# Meet Meridian Corporation

*An industrial supplier with robot-run warehouses, all on one Salesforce org*

### Sales

Runs the CRM: opportunities and quotes

*Built with an outside consultancy*

### Service

Cases, complaints and returns

*Built by a second vendor*

### Marketing

Campaigns and customer engagement

*A small in-house team*

### Meridian Fulfillment

The Warehouse Operations app: North Hub and its robots

*The app from Sessions 2 to 5*

For years it worked, because nobody's code touched anyone else's.

# Then the divisions need each other

*And the org starts turning into Happy Soup*

### Sales → Fulfillment

A closed deal should create a fulfillment order

### Service → Fulfillment

Tell a customer where their order is

### Marketing → everyone

Its fields on every Account query

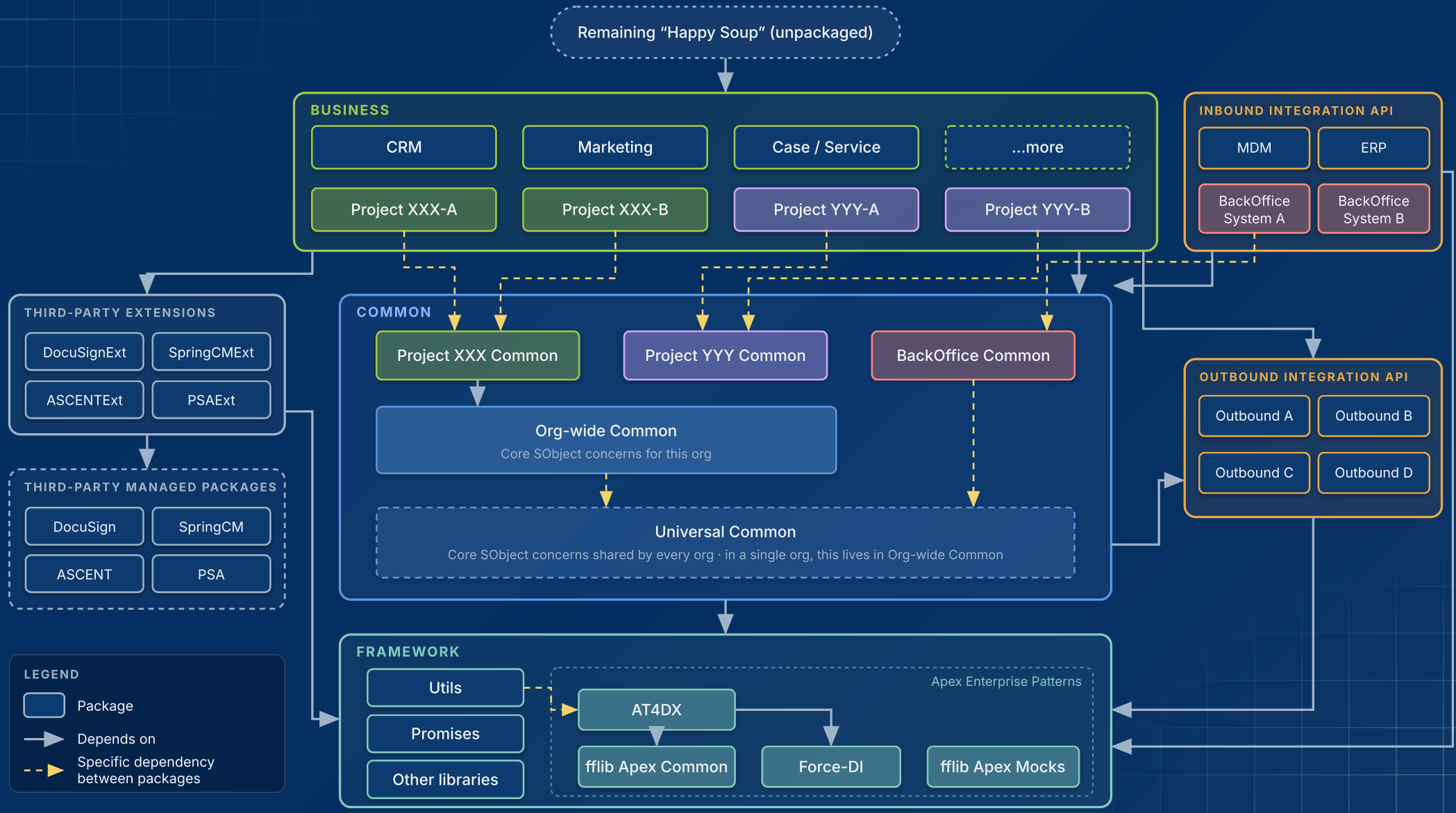
⚠️ **Four teams edit the same Account trigger and the same Account Selector.**
⚠️ **Every release needs everyone.** One vendor's change breaks another vendor's tests.

# Leadership's question

> *How do the divisions work together without wiring themselves so tightly that every change ripples through all of them?*

The question from today's abstract

# The package layers



# What lives in each layer

*Top to bottom*

| Layer | What lives here | Rule |
|---|---|---|
| **Remaining "Happy Soup"** | Metadata not yet packaged | Shrinks over time; depends on everything below it |
| **Business** | One package per division or project: CRM, Service, Marketing, project packages | Owns its own objects, fields and processes. Never depends on another business package |
| **Project Common** | Shared by one project's business packages | Exists only when two or more business packages need the same thing |
| **Org-wide Common** | Core SObject concerns for this org | Owns the shared objects and their one Selector and one Domain |
| **Universal Common** | Core SObject concerns common to all of the company's orgs | Multi-org only. In a single org, its contents live in Org-wide Common |
| **Framework** | fflib Apex Common and Apex Mocks, force-di, AT4DX, utils | No company schema at all |

Beside the stack: integration API packages on common, and third-party extension packages on managed packages.

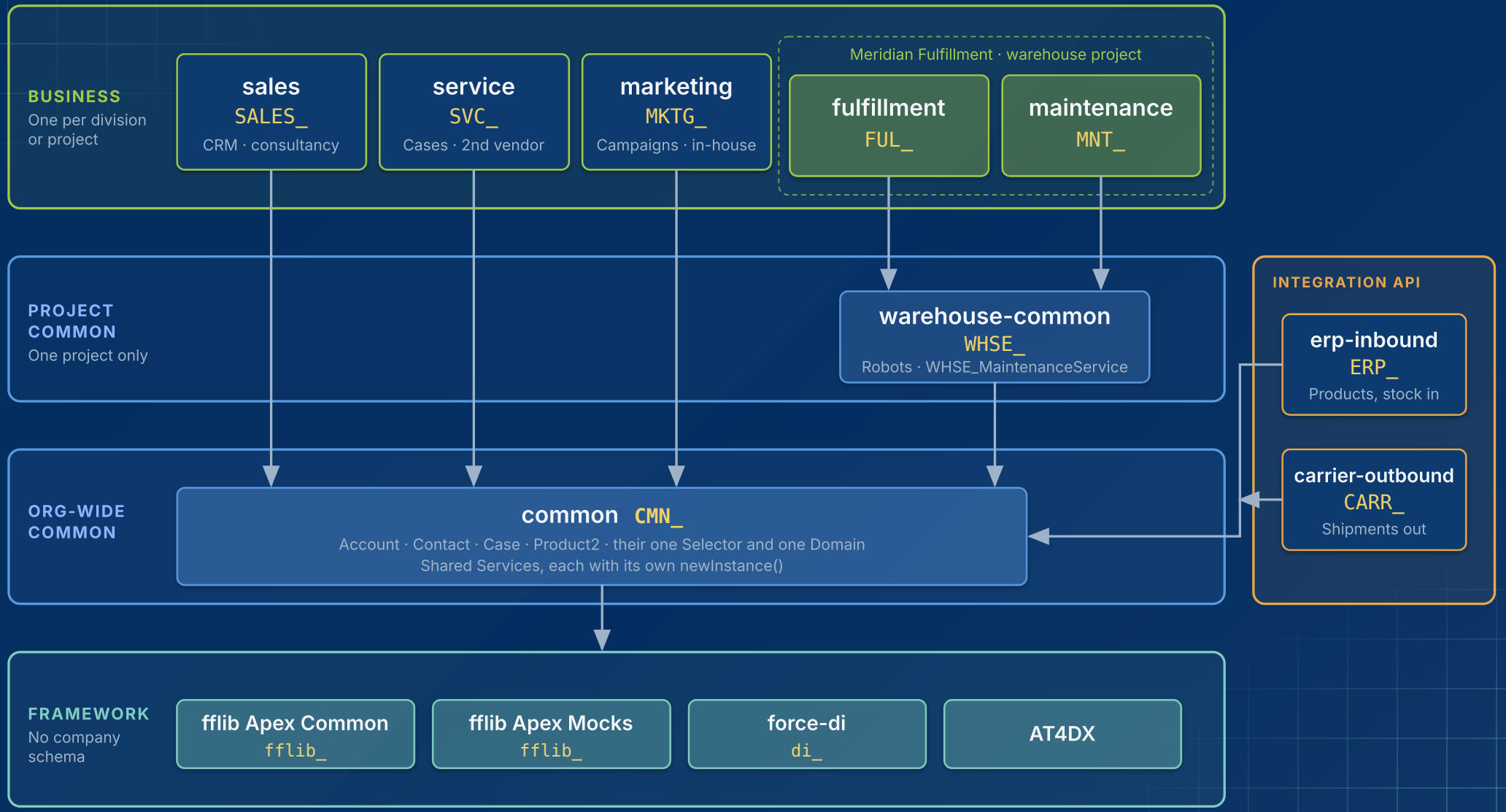
# Three rules

1. **Dependencies only point down**
   A lower layer never names anything in a higher one

2. **Each object has one owner**
   The lowest layer that needs it owns it. Higher layers extend it through injection, not by editing it

3. **Push shared things down only as far as needed**
   Two business packages share something? It goes in their Project Common, not Org-wide Common

# Meridian on the map



# One org, or many

*Meridian is a single-org story. The pattern doesn't change with more orgs*

- **Single org:** anything that would live in Universal Common goes into that org's Org-wide Common package
- **Multi-org:** it moves down into a Universal Common package, prefix `UCMN_`, that every org installs
- **Everything above it is the same:** business packages, Project Common, contracts and bindings

> Start in one org. If a second org arrives, move shared pieces down a layer.

# 1 · Cross-package architecture

*FFLib's layers already give you the boundaries. Packages make the platform enforce them*

## Why split at all

- Teams ship on different schedules
- The org is too big for one deploy
- Some features belong to one division or region

## What unlocked packages give you

- Each team gets its own version, tests and release
- The platform checks the dependencies
- Ownership is visible, not tribal knowledge

## Where FFLib lands

- **Framework:** fflib, force-di, AT4DX
- **Common:** shared objects with their Selector and Domain, and shared Services
- **Business:** its own Domains, Selectors, Services, registered through custom metadata

# Dependencies only point down

*The rule that forces everything else today*

- `common` **can never name a class in** `sales` **or** `fulfillment` **.** It doesn't compile, because it can't see them
- `sales` **can't name a class in** `fulfillment` **either.** Business packages never depend on each other
- **So when a lower package needs work done above it, it can't call it directly.** It has to ask something for it

> If you can't call it, you ask for it. Topics 2 to 4 are the ways to ask.

# Package prefixes, not namespaces

*Framed for an enterprise: 2GP unlocked packages, no namespace*

- **Each package gets a prefix:** 3 to 8 capital letters, an underscore, then the element's proper name
- `SALES_OpportunitiesService`, `SVC_SLATier__c`, `WHSE_Robot__c`: the prefix says which package owns it
- **Session 5's `WDGT_` prefix was a package prefix inside one package.** Today every package gets one

> Namespaces and `@NamespaceAccessible` are for ISVs building managed packages. The AEP frameworks support that too; it's not today's case.

# Naming conventions

*"Let the metadata's API name tell you where it belongs"*

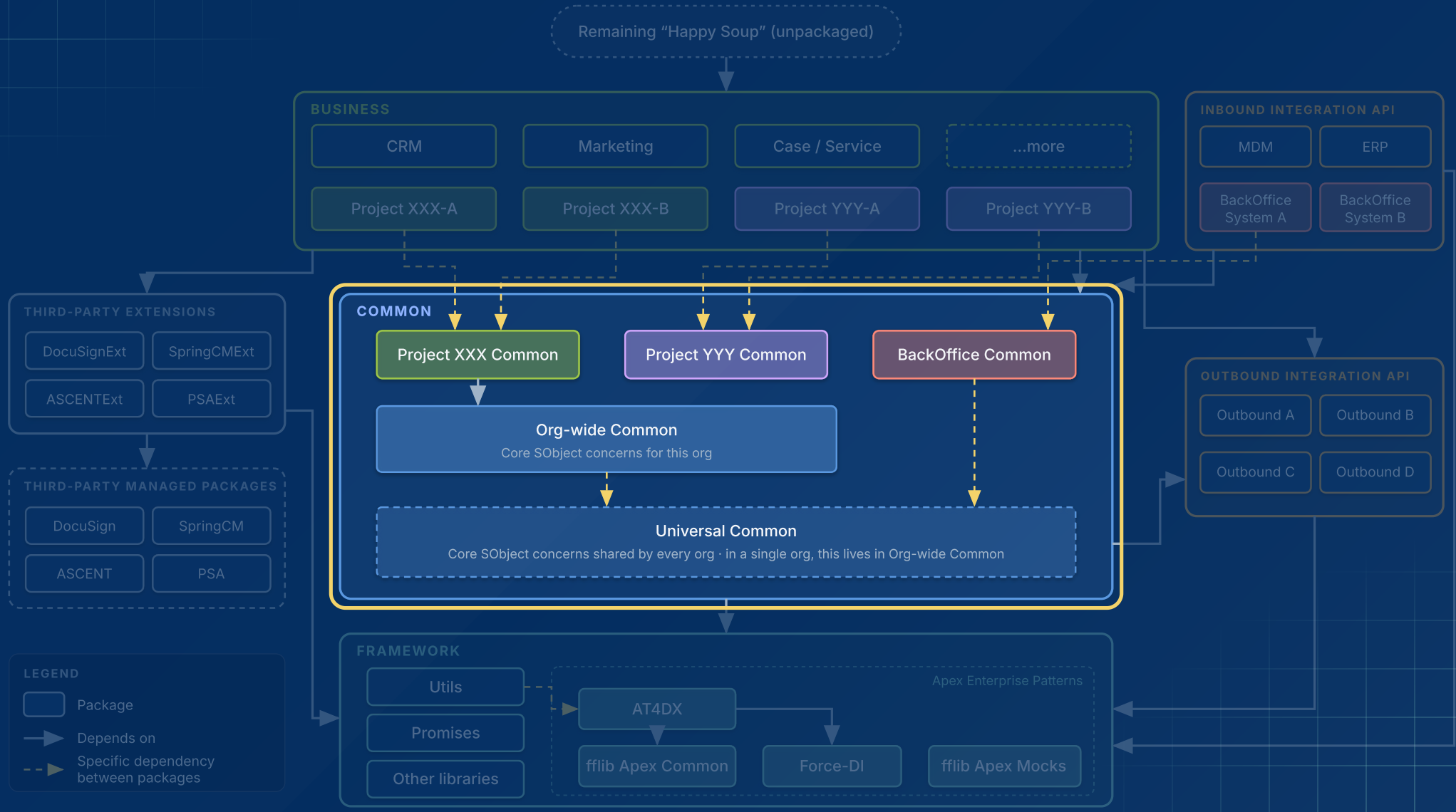
A name has two parts: the **prefix** says which package owns it, and the **suffix** says what kind of artifact it is.

| Name | Owner | Artifact |
|---|---|---|
| `SALES_AccountsSelector` | sales | Selector |
| `WHSE_IMaintenanceService` | warehouse-common | Service interface |
| `WHSE_MaintenanceServiceImpl` | warehouse-common | Service implementation |
| `SVC_CaseController` | service | Controller |
| `FUL_FulfillmentRestAPI` | fulfillment | REST API |
| `MKTG_EngagementScore__c` | marketing | Custom field on Account |

> Pays off in code review, in audits, and while code is still moving out of the Happy Soup.

# 2 · Shared contracts and interfaces

*Shared Services live in a lower layer, so business packages use them without depending on each other*



# Interfaces return, on purpose

*Session 5 saved this case for today*

- **Session 5:** use interfaces "only where implementations genuinely vary"
- **Across packages, they can vary:** a higher package can swap in its own implementation with a binding
- **Each Service ships as a triad in one package:** the facade, the interface and the implementation
- **Callers use the facade's** `newInstance()` **,** and only ever hold the interface. They never name the implementation

> Andy, Session 2: interfaces are how Services get resolved. Here's where it matters.

# One Service, three classes, one package

*Selectors and Domains follow the same rule*

## Ships together

- `WHSE_MaintenanceService`: the facade, with a static `newInstance()`
- `WHSE_IMaintenanceService`: the interface, with the records and simple data classes it passes
- `WHSE_MaintenanceServiceImpl`, and its binding record

## Every consumer

- Calls `WHSE_MaintenanceService.newInstance()`, from any package
- Holds the interface type, never the implementation
- Never calls `Application.Service` itself: only `newInstance()` does

## Keep it stable

- Every change moves every package that calls it
- Add methods; avoid changing or removing old ones
- Put the triad in the lowest package its callers depend on, and no lower

**Selectors and Domains too:** each has its own static `newInstance()`, such as `CMN_AccountsSelector.newInstance()`, and every consumer calls it.

# Fulfillment calls Maintenance

*Both warehouse packages need maintenance scheduling, so it moves down to warehouse-common*

**FulfillmentService.cls · Sessions 2 to 5**

```
// One package: it can see MaintenanceService
private final MaintenanceService maintenanceSvc;

public static FulfillmentService newInstance() {
  return new FulfillmentService(
    // ...selectors
    MaintenanceService.newInstance()
  );
}

// Hand due robots to maintenance
maintenanceSvc.scheduleService(dueRobots, uow);
```

**FUL_FulfillmentService.cls · today**

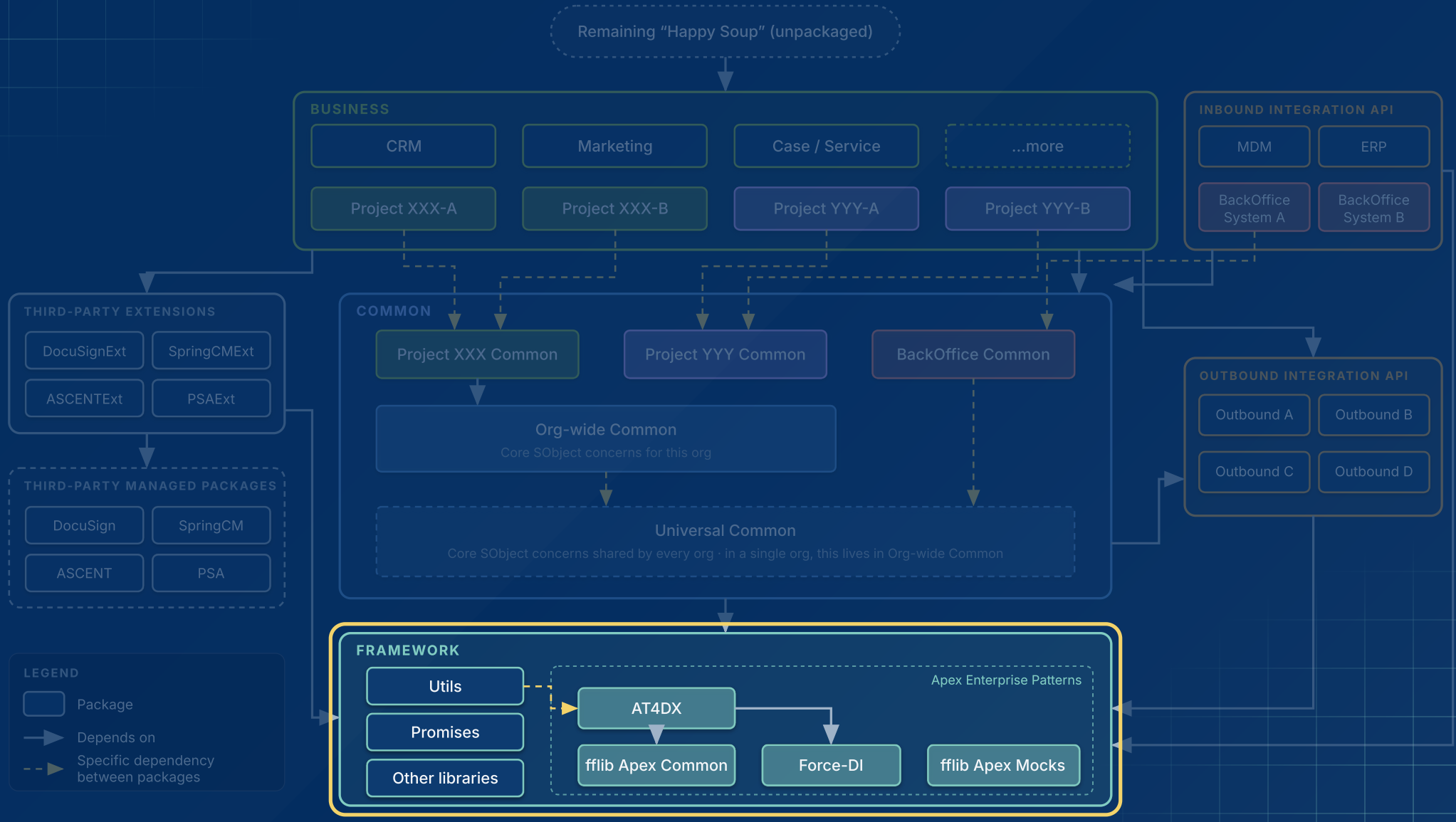
```
// Scheduling now lives in warehouse-common
private final WHSE_IMaintenanceService maintenanceSvc;

public static FUL_FulfillmentService newInstance() {
  return new FUL_FulfillmentService(
    // ...selectors
    WHSE_MaintenanceService.newInstance()
  );
}

// Same call, through the interface
maintenanceSvc.scheduleService(dueRobots, uow);
```

# 3 · Configuration-driven design

*How the factory picks an implementation, and lets a higher package replace it*



# Why a factory, across packages

*"AT4DX keeps a factory, on purpose"*

- **Session 5's `newInstance()` stays:** every Service, Selector and Domain has one, and every consumer calls it
- **Inside it, AT4DX asks the factory:**
  `Application.Service.newInstance(WHSE_IMaintenanceService.class)`
- **The factory reads the bindings,** so a higher package can replace the implementation without touching any caller
- **There's no app-specific `Application` class to edit:** AT4DX's `Application` lives in the framework and reads bindings from every installed package

> Callers call `newInstance()`. It asks the factory. Custom metadata decides the class. Tests still `setMock()`.

# Four factories, four bindings

*Bindings live in custom metadata, and deploy with the package that owns the class*

| Factory | Custom metadata | Key fields |
|---|---|---|
| `Application.Selector` | `ApplicationFactory_SelectorBinding__mdt` | `BindingSObject__c`, `To__c`, `Priority__c` |
| `Application.Domain` | `ApplicationFactory_DomainBinding__mdt` | `BindingSObject__c`, `To__c` (a Domain's constructor class) |
| `Application.Service` | `ApplicationFactory_ServiceBinding__mdt` | `BindingInterface__c`, `To__c`, `Priority__c` |
| `Application.UnitOfWork` | `ApplicationFactory_UnitOfWorkBinding__mdt` | `BindingSObject__c`, `BindingSequence__c` |

Installing a package adds its bindings, and a higher `Priority__c` replaces one. Under the covers it's force-di: metadata-driven dependency injection.

# A binding record

*Story: Service asks where an order is*

**`fulfillment`** owns order status: `FUL_OrderFulfillmentService`, `FUL_IOrderFulfillmentService` and `FUL_OrderFulfillmentServiceImpl`, plus this record:

| ApplicationFactory_ServiceBinding__mdt | FUL_OrderFulfillmentService |
|---|---|
| BindingInterface__c | `FUL_IOrderFulfillmentService` |
| To__c | `FUL_OrderFulfillmentServiceImpl` |
| Priority__c | 1 |

> service can't depend on fulfillment, so fulfillment ships the "Where's my order?" component for the Case page. service never names a `FUL_` class.

# Asking the factory

*Only the facade talks to the factory. Every caller uses* `newInstance()`

FUL_OrderFulfillmentService.cls · fulfillment · the facade

```apex
public inherited sharing class FUL_OrderFulfillmentService {
  public static FUL_IOrderFulfillmentService newInstance() {
    return (FUL_IOrderFulfillmentService)
      Application.Service.newInstance(FUL_IOrderFulfillmentService.class);
  }
}
```

FUL_OrderStatusController.cls · a consumer

```apex
List<FUL_OrderStatus> statuses = FUL_OrderFulfillmentService.newInstance()
  .getStatusForAccounts(accountIds);
```

FUL_OrderStatusControllerTest.cls · a test

```apex
// The mock beats any binding (same as Session 5)
Application.Service.setMock(FUL_IOrderFulfillmentService.class, mockService);
```

A regional package can ship a higher-priority binding with its own implementation. No caller changes.

# Overrides, and a caution

*Configuration is powerful, and it hides the call graph*

## Overrides without forks

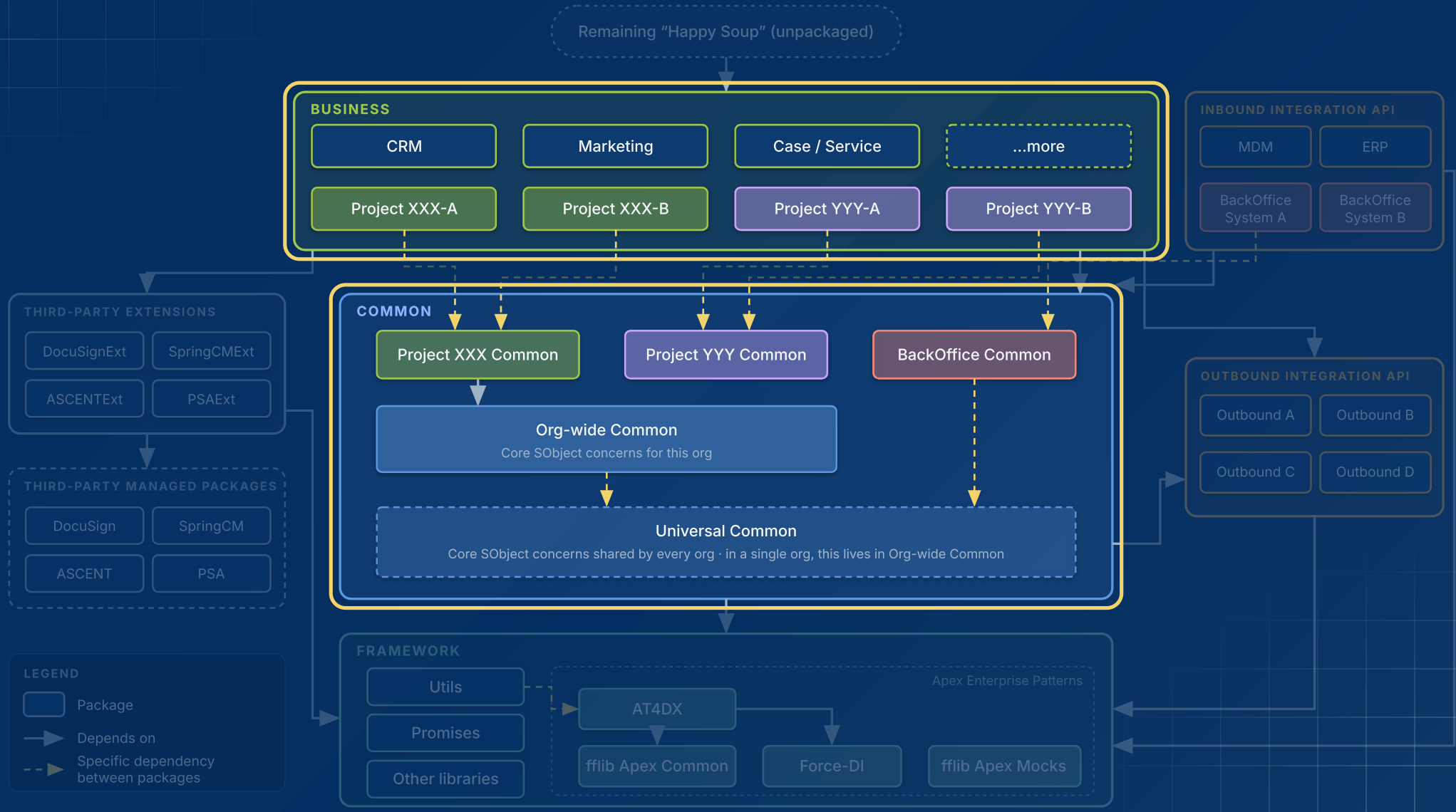
- A later package, or a subscriber org, supplies a different binding for the same interface
- `Priority__c` decides which one wins
- A regional package can replace one Service without touching the original

## Keep it traceable

- Name bindings clearly, with the package prefix
- Keep one binding per interface unless you mean to override
- Add a test that checks every binding resolves

# 4 · Extensibility patterns

*How a higher layer adds fields, queries and logic to objects a lower layer owns*



# Five AT4DX techniques

*Each FFLib layer has an extension point*

| Technique | Extension point | What a higher-layer package adds |
|---|---|---|
| **Application factory injection** | `ApplicationFactory_*Binding__mdt` | Its own Selector, Domain or Service, or an override of one |
| **Selector field injection** | `SelectorConfig_FieldSetInclusion__mdt` | Its own fields on another package's queries, through a field set |
| **Selector method injection** | `selectInjection(...)` on the shared Selector | Its own query against an object a lower layer owns |
| **Domain process injection** | `DomainProcessBinding__mdt`, plus criteria and action classes | Logic inside another package's Domain, in a set order |
| **Platform event distribution** | `PlatformEvents_Subscription__mdt`, plus an `IEventsConsumer` | A subscriber to another package's events |

# Selector field injection

*Story: Marketing adds fields to Account*

### common owns

`Account` and `CMN_AccountsSelector`, which extends `ApplicationSObjectSelector` with `includeFieldSetFields` on

### marketing ships

- `MKTG_Slogan__c`, `MKTG_EngagementScore__c`
- A field set: `MKTG_AccountFields`
- A `SelectorConfig_FieldSetInclusion__mdt` record: `BindingSObject__c` = Account, `FieldsetName__c` = `MKTG_AccountFields`

### Result

Every Account query through the Selector comes back with marketing's fields

common never changes

**"The core package never learns that the marketing package exists, yet Account queries come back with the marketing fields."**

# Selector method injection

*Story: Service needs accounts with more than five open complaint Cases*

SVC_AccountsWithOpenComplaintsSelectorMethod.cls · service

```
1  public inherited sharing class SVC_AccountsWithOpenComplaintsSelectorMethod
2    extends AbstractSelectorMethodInjectable
3    implements ISelectorMethodInjectable {
4
5    public List<SObject> selectQuery() {
6      Parameters params = (Parameters) getParams();
7      Integer minimum = params.minimumOpenComplaints;
8      // newQueryFactory() is the shared Selector's: its fields, field sets and security
9      return Database.query(newQueryFactory()
10       .setCondition('SVC_OpenComplaintCount__c > :minimum').toSOQL());
11   }
12
13   public class Parameters implements ISelectorMethodParameterable {
14     public Integer minimumOpenComplaints;
15   }
16 }
```

SVC_ComplaintsServiceImpl.cls · a caller in service

```
1 SVC_AccountsWithOpenComplaintsSelectorMethod.Parameters params =
2   new SVC_AccountsWithOpenComplaintsSelectorMethod.Parameters();
3 params.minimumOpenComplaints = 5;
4
5 List<Account> accounts = CMN_AccountsSelector.newInstance()
6   .selectInjection(SVC_AccountsWithOpenComplaintsSelectorMethod.class, params);
```

# Domain process injection

*Story: Service reacts to a late shipment*

### common owns

The `Case` Domain, extending `ApplicationSObjectDomain`. Its trigger runs any injected processes for each trigger event

### service ships

- **Criteria:** `SVC_LateShipmentCriteria` (`IDomainProcessCriteria`) keeps complaint Cases about late orders
- **Action:** `SVC_FlagAccountAction` (`IDomainProcessAction`) flags their Accounts

### Two `DomainProcessBinding__mdt` records

- `RelatedDomainBindingSObject__c` = Case
- `TriggerOperation__c` = After_Insert
- `Type__c` = Criteria, then Action
- `OrderOfExecution__c` sets the order

**Session 3 drew the line between the Domain and the trigger handler.** Domain process injection is that same line, opened to other packages.

# Platform event distribution

*Story: Sales closes a deal, and the warehouse ships it*

**sales: when an Opportunity closes**

```
AT4DXMessage__e evt = new AT4DXMessage__e();
evt.Category__c  = 'Opportunity';
evt.EventName__c = 'SALES_OPPORTUNITY_CLOSED_WON';
evt.Payload__c   = JSON.serialize(opportunityIds);
EventBus.publish(evt);
```

**FUL_ClosedWonConsumer.cls · fulfillment**

```
public class FUL_ClosedWonConsumer
  extends PlatformEventAbstractConsumer {
  public override void runInProcess() {
    Set<Id> oppIds = new Set<Id>();
    for (SObject sobj : events) {
      AT4DXMessage__e evt = (AT4DXMessage__e) sobj;
      oppIds.addAll((Set<Id>) JSON.deserialize(
        evt.Payload__c, Set<Id>.class));
    }
    FUL_FulfillmentOrdersService.newInstance()
      .createFor(oppIds);
  }
}
```

fulfillment ships a `PlatformEvents_Subscription__mdt` record: `EventBus__c`, `EventCategory__c`, `Event__c`, `Consumer__c`. `PlatformEventDistributor` hands matching events to the consumer.

# Events decouple the most

*Pick the lightest tool that does the job*

## Why events

- The publisher doesn't know who listens
- No interface to keep stable, only an event name and payload
- Add a subscriber by installing a package

## The trade-off

- Asynchronous: the deal closes before the order exists
- Harder to trace and to debug
- Document event names like a contract, because they are one

Need an answer now? Call the owning Service (topic 2). Telling others something happened? Publish an event.

# Meridian's stories, on the map

*Every division works with the others, and no business package depends on another*

| Story | From → to | Technique | Lives in |
|---|---|---|---|
| Sales closes a deal, the warehouse ships it | sales → fulfillment | Platform event distribution | fulfillment |
| Service asks where an order is | fulfillment → Case page | Service triad + binding | fulfillment |
| Fulfillment schedules robot maintenance | fulfillment → warehouse-common | Service triad + binding | warehouse-common |
| Marketing adds fields to Account | marketing → common | Selector field injection | marketing |
| Service reacts to a late shipment | service → common | Domain process injection | service |
| Service queries its own Accounts | service → common | Selector method injection | service |

# Does it work at scale?

*One enterprise client, built on these patterns*

**120**
deployments a month, on average

**60+**
applications

**20+**
app teams

**What made it possible:** layered packages, one owner per object, Services called through `newInstance()`, and custom metadata deciding the wiring. Teams ship on their own schedule without waiting on each other.

# Depend on contracts. Let configuration decide.

## Promises kept

| | |
|---|---|
| ✅ Metadata-driven dependency injection | Andy, S2 |
| ✅ Interfaces to resolve Services | Andy, S2 |
| ✅ Selector fields from another package | Session 4 |
| ✅ `@NamespaceAccessible` : it's for ISVs | Session 4 |
| ✅ `WDGT_` , across real packages | Session 5 |
| ✅ AT4DX keeps a factory, on purpose | Session 5 |

## Remember the map

**Dependencies only point down.**

**Each object has one owner.**

**Push shared things down only as far as needed.**

*Thank you, Sally, Andy and John Storey, for the series.*

# Likely questions

*Short answers to start from*

### Do I need AT4DX?

No. You can pass interfaces and use a small factory of your own. AT4DX pays off once several packages extend the same objects.

### Migrating an org?

Pull shared objects and Services into common first. Then move one feature at a time, with every call going through `newInstance()`.

### Is `Application` required now?

Session 5 said it's optional in one package. Across packages, the factory is the point.

### Harder to debug?

Somewhat. Clear binding names, a test that every binding resolves, and rare overrides.

### Performance?

Custom metadata reads don't count against the SOQL limit. Load bindings once per transaction anyway.

### Namespaces?

An ISV's concern with managed packages, and supported. Enterprises use unlocked packages and prefixes.

# Learn more

*Where to go next*

| Resource | Where |
| --- | --- |
| **Apex Enterprise Patterns** (all the framework repos) | github.com/apex-enterprise-patterns |
| **fflib Apex Common** | github.com/apex-enterprise-patterns/fflib-apex-common |
| **fflib Apex Mocks** | github.com/apex-enterprise-patterns/fflib-apex-mocks |
| **force-di** | github.com/apex-enterprise-patterns/force-di |
| **AT4DX** | github.com/apex-enterprise-patterns/at4dx |
| **AT4DX sample code** | github.com/apex-enterprise-patterns/at4dx-samplecode |
| **Brewtiful Beans Learns AT4DX**, a blog series by **Jake Richter** (the same ideas, with a coffee roaster) | syntax-syllogism.com/blog/tags/brewtiful-beans |
| **This series**, slides for all six sessions | github.com/CodeWithSally/CodeWithSally-Apex-English/Sessions/FFLibSeries |

# Thank you

## Questions?

FFLib Series · Session 006

John M. Daniel

github.com/ImJohnMDaniel

x.com/ImJohnMDaniel

linkedin.com/in/ImJohnMDaniel

